

Question Bank

SSAD and SE

Five Mark Questions

1. Distinguish between alpha testing, beta testing and acceptance testing ?

Ans :-

Testing Type	Performed By	Environment	Purpose
Alpha Testing	Developers or QA team	In-house (developer's site)	To find bugs before releasing to real users
Beta Testing	End users / selected customers	Real environment (user's site)	To get feedback from actual users and identify remaining issues
Acceptance Testing	Customer / client	Real environment	To verify system meets requirements and approve for deployment

2. Find the boundary value test suite for a function that computes the square root of the integer values in the range of 0 to 5000. (2+3)?

Ans :-

Step 1: Identify the boundaries

The **input range** is: **0 to 5000**

For **boundary value analysis (BVA)**, we test:

- **Minimum value (min):** 0
- **Just above minimum (min + 1):** 1
- **Just below maximum (max - 1):** 4999
- **Maximum value (max):** 5000

We also include **values just outside the boundaries** to test error handling:

- **Below minimum (min - 1):** -1 (invalid input)
- **Above maximum (max + 1):** 5001 (invalid input)

Step 2: Construct the boundary value test suite

Test Case	Input	Expected Behavior
TC1	-1	Error / Invalid input
TC2	0	$\sqrt{0} = 0$
TC3	1	$\sqrt{1} = 1$
TC4	4999	$\sqrt{4999} \approx 70.70$
TC5	5000	$\sqrt{5000} \approx 70.71$
TC6	5001	Error / Invalid input

3. Design the equivalence class test cases for a program that reads two integer pairs (m_1, c_1) and (m_2, c_2) defining two straight lines of the form $y = mx + c$. The program computes the intersection point of the two straight lines and displays the point of intersection ?

Ans :-

Problem Analysis

We have a program that reads two lines:

Line 1: $y = m_1x + c_1$ Line 2: $y = m_2x + c_2$

Key conditions for intersection:

- I. **Lines are parallel** $\rightarrow$ No intersection $\rightarrow$ slopes are equal ($m_1 = m_2$)
- II. **Lines coincide** $\rightarrow$ Infinite intersections $\rightarrow$ slopes and intercepts equal ($m_1 = m_2$ and $c_1 = c_2$)
- III. **Lines intersect at one point** $\rightarrow$ Slopes are different ($m_1 \neq m_2$)

We can create **equivalence classes** based on these conditions.

Step 1: Identify Equivalence Classes (EC)

EC ID	Description	Example Input
EC1	Lines intersect at one point (valid)	$(m_1 = 2, c_1 = 3; m_2 = 1, c_2 = 4)$
EC2	Lines are parallel (no intersection)	$(m_1 = 2, c_1 = 3; m_2 = 2, c_2 = 5)$
EC3	Lines coincide (infinite intersections)	$(m_1 = 2, c_1 = 3; m_2 = 2, c_2 = 3)$
EC4	Invalid input (non-integer / missing)	$(m_1 = 'a', c_1 = 3; m_2 = 2, c_2 = 4)$

Step 2: Select Test Cases (One from each EC)

Test Case	m1	c1	m2	c2	Expected Result
TC1	2	3	1	4	Intersection point ((x, y))
TC2	2	3	2	5	No intersection (parallel lines)
TC3	2	3	2	3	Infinite intersections (coincident lines)
TC4	'a'	3	2	4	Error / Invalid input

4. Find the effort required to develop the product, development time and cost to develop the product(follow Basic COCOMO),assuming that the product is of organic type, the size of the software product has been estimated to be 32000 LOC and the average salary of software developers is Rs. 15000 per month. (2+2+1)?

Ans :-

Given:

- Project type = **Organic**
- Size = **32,000 LOC = 32 KLOC**
- Average salary = Rs. 15,000/month

Basic COCOMO formulas for Organic type:

a. Effort (in person-months):

$$\text{Effort (PM)} = 2.4 \times (\text{KLOC})^{1.05}$$

b. Development Time (TDEV in months):

$$\text{TDEV} = 2.5 \times (\text{Effort})^{0.38}$$

c. Cost (Rs.)

$$\text{Cost} = \text{Effort} \times \text{Average salary}$$

Step 1: Effort

$$\text{Effort} = 2.4 \times (32)^{1.05}$$

First, calculate $32^{1.05}$:

- $\ln 32 \approx 3.4657$
- $32^{1.05} = e^{1.05 \cdot 3.4657} = e^{3.639} \approx 38$

Now, Effort:

$$\text{Effort} = 2.4 \times 38 \approx 91.2 \text{ person-months}$$

Effort $\approx$ 91 PM

Step 2: Development Time (TDEV)

$$\text{TDEV} = 2.5 \times (91.2)^{0.38}$$

- $91.2^{0.38} \approx 5.24$ (approx)
- $\text{TDEV} = 2.5 \times 5.24 \approx 13.1$ months

Development Time $\approx$ 13 months

Step 3: Cost

$$\text{Cost} = \text{Effort} \times \text{Salary} = 91.2 \times 15,000 \approx 1,368,000 \text{ Rs.}$$

Cost $\approx$ Rs. 13.68 lakh

Answer (Summary):

- **Effort:** 91 person-months
- **Development Time:** 13 months
- **Cost:** Rs. 13,68,000

5. Identify how we distinguish between organic, semi detached and embedded type of software project in Basic COCOMO ?

Ans :-

Distinguishing Software Project Types in Basic COCOMO:

In **Basic COCOMO**, software projects are classified into **three types** based on **project size, complexity, team experience, and constraints**:

Project Type	Characteristics	Example
Organic	- Small, simple projects- Small team with good experience- Flexible requirements- Few constraints	Payroll system, simple inventory system

Project Type	Characteristics	Example
Semi-Detached	- Medium-sized projects- Mixed experience team- Some rigid requirements and moderate complexity	Banking system, airline reservation system
Embedded	- Large, complex projects- Strong hardware, software, and operational constraints- Real-time or safety-critical	Air traffic control system, medical devices, spacecraft software

Key factors to distinguish:

- I. **Project size** (LOC/KLOC)
- II. **Team experience** (small & expert vs. large & mixed)
- III. **Requirements** (flexible vs. rigid)
- IV. **Constraints** (hardware, software, timing).

6. Discuss at least five different types of coupling that might exist between two modules (Give examples of each)?

Ans :-

Coupling measures **how dependent one module is on another**. **Low coupling** is desirable. Here are **five main types**:

Type of Coupling	Description	Example
Content Coupling	One module directly uses the content (e.g., code, data) of another module. Very high dependency	Module A directly modifies a variable inside Module B
Common Coupling	Two modules share the same global data	Module A and Module B both use a global variable counter
Control Coupling	One module controls the behavior of another by passing control information (flags)	Module A calls Module B with a flag: processData(mode=1)

Type of Coupling	Description	Example
Stamp (Data-structured) Coupling	One module passes a data structure , but the called module uses only part of it	Module A sends a full Employee object to Module B, but B only uses Employee.name
Data Coupling	Modules share only the necessary data (parameters), no extra info	Module A calls Module B with calculateArea(length, width)

7. Compare white-box testing with black-box testing with examples of each.

Ans :-

Feature	White-Box Testing	Black-Box Testing
Definition	Tests the internal structure, logic, and code of the program	Tests the functionality of the software without knowing the internal code
Tester Knowledge	Requires knowledge of programming and code	Does not require programming knowledge
Focus	Code coverage, paths, branches, loops	Inputs, outputs, and functional requirements
Example	Testing all branches of an if-else statement in code	Testing a login form with correct and incorrect username/password combinations
Purpose	Detect logical errors, missing paths, and code flaws	Verify that the software meets user requirements
Techniques	Statement coverage, branch coverage, path coverage	Equivalence partitioning, boundary value analysis, decision table testing

8. Determine the Unadjusted Function Point (UFP) using the below information about a software project being developed with the following functional components:

Component Type	Count	Complexity	Weighting Factor
External Inputs (EI)	10	Simple	3
External Outputs (EO)	5	Average	5
External Inquiries (EQ)	8	Complex	6
Internal Files (ILF)	4	Complex	15
External Files (EIF)	6	Average	7

Ans :-

Component Type	Count	Complexity	Weighting Factor
External Inputs (EI)	10	Simple	3
External Outputs (EO)	5	Average	5
External Inquiries (EQ)	8	Complex	6
Internal Files (ILF)	4	Complex	15
External Files (EIF)	6	Average	7

Step 1: Multiply Count × Weighting Factor for each component

- I. **EI:** $10 \times 3 = 30$
- II. **EO:** $5 \times 5 = 25$
- III. **EQ:** $8 \times 6 = 48$
- IV. **ILF:** $4 \times 15 = 60$
- V. **EIF:** $6 \times 7 = 42$

Step 2: Add all results to get UFP

$$\text{UFP} = 30 + 25 + 48 + 60 + 42 = 205$$

Answer: Unadjusted Function Point (UFP) = 205

9. Discuss different types of maintenance required for a typical software product?

Ans :- Software maintenance is the **modification of a software product after it has been delivered** to correct faults, improve performance, or adapt it to a changed environment.

Here are the **main types of maintenance**:

I. Corrective Maintenance

- **Purpose:** Fix **errors or defects** discovered after delivery.
- **Example:** Correcting a bug in a billing system that caused wrong invoice calculations.

II. Adaptive Maintenance

- **Purpose:** Modify software to **work in a changed environment** (hardware, OS, software upgrades).
- **Example:** Updating a program to run on a new version of Windows.

III. Perfective Maintenance

- **Purpose:** Improve **performance, maintainability, or usability** without changing functionality.
- **Example:** Optimizing a search algorithm to reduce execution time.

IV. Preventive Maintenance

- **Purpose:** Make changes to **prevent future problems** or reduce system vulnerabilities.
- **Example:** Refactoring code to improve structure and reduce risk of future bugs.

10. Write the main disadvantages of a classical waterfall model?

Ans :-

Main Disadvantages of the Classical Waterfall Model (SSAD / SE):

I. Rigid and Inflexible:

- Once a phase is completed, it's **difficult to go back** and make changes.

II. Poor Adaptation to Changes:

- Requirements **must be clearly defined upfront**; any change during development is costly.

III. Late Testing:

- Testing happens **only after coding**, so defects are discovered **late**, increasing cost and effort.

IV. **Not Suitable for Large/Complex Projects:**

- For projects with evolving requirements, the Waterfall model **fails to handle uncertainty**.

V. **Customer Feedback Delayed:**

- Customers **see the product only at the end**, reducing chances for early corrections.

11. Write at least five main disadvantages of using LOC as project size estimation metric.

Ans :-

Disadvantages of Using Lines of Code (LOC) as a Project Size Estimation Metric:

I. **Language Dependency:**

- LOC varies with programming language; the same functionality may require **more lines in one language** than another.

II. **Ignores Complexity:**

- LOC counts lines but does **not measure logic complexity or difficulty** of the code.

III. **Encourages Inefficient Coding:**

- Developers may write **more lines of code than necessary** to increase LOC count.

IV. **Does Not Account for Reuse:**

- LOC metric does not consider **reused modules or libraries**, potentially **overestimating size**.

V. **Not Suitable Early in Project:**

- At the requirements or design stage, LOC **cannot be accurately estimated**, making early effort prediction unreliable.

VI. **Measurement Inconsistency: (Extra)**

- Counting LOC can vary depending on **coding style, comments, and blank lines**, leading to inconsistent estimates.

12. Discuss at least five different types of cohesion that might exist in a module (give examples of each)?

Ans :- Cohesion measures **how closely the elements within a module are related**. **High cohesion** is desirable. Here are **five main types**:

Type of Cohesion	Description	Example
Functional Cohesion	All parts of the module work together to	A module calculateSalary() that computes employee

Type of Cohesion	Description	Example
	perform a single well-defined task	salary based on hours, rate, and deductions
Sequential Cohesion	Output of one part is input to the next part within the module	A module that reads data → processes it → writes results in sequence
Communicational Cohesion	All parts operate on the same data set	A module that calculates average, max, min of a student marks file
Procedural Cohesion	Parts are related by sequence of execution , not by a single task	A module that prints reports in multiple formats sequentially
Temporal Cohesion	Parts are executed together at the same time , usually initialization or cleanup	A module that initializes system settings at startup
Logical Cohesion (extra)	Performs related functions , but one function is selected at runtime	A module processInput() that handles keyboard, mouse, or touchscreen input depending on user choice
Coincidental Cohesion (extra)	Parts are unrelated , grouped arbitrarily	A module that performs printing, logging, and file cleanup together — undesirable!

Ten Mark Questions

1. (a) Discuss statement coverage, branch coverage and condition coverage with respect to white box testing. (2+2+2)

(b) Write the advantages and disadvantages of prototyping model for software development. (2+2)?

Ans :- (a) **White-Box Testing Coverage Types**

I. Statement Coverage

- **Definition:** Ensures **every executable statement in the code** is executed at least once during testing.
- **Example:** In a simple if statement, each line of code should run at least once.
- **Goal:** Detect **unreachable code or missing execution paths**.

II. Branch Coverage (Decision Coverage)

- **Definition:** Ensures **every possible branch (true/false) of each decision point** is executed at least once.
- **Example:** For if ($x > 0$), test with $x > 0$ and $x \leq 0$.
- **Goal:** Detect **errors in conditional logic** that may not be caught by statement coverage.

III. Condition Coverage (Predicate Coverage / Multiple Condition Coverage)

- **Definition:** Ensures **all atomic conditions in a decision** are evaluated to both true and false at least once.
- **Example:** For if ($x > 0 \ \&\& \ y < 10$), test combinations where $x > 0$ true/false and $y < 10$ true/false.
- **Goal:** Detect **logical errors in individual conditions**.

(b) Prototyping Model: Advantages & Disadvantages

Advantages:

- A. Helps **clarify unclear requirements** by showing a working model to the user.
- B. Encourages **user feedback**, reducing the risk of developing unwanted features.

Disadvantages:

- A. Can lead to **poor design** if prototype is rushed and used as final product.
- B. **Costly and time-consuming** if many iterations of the prototype are needed.

2. Draw and explain context diagram and 1-level DFD for “Online Examination System”?

Ans :-

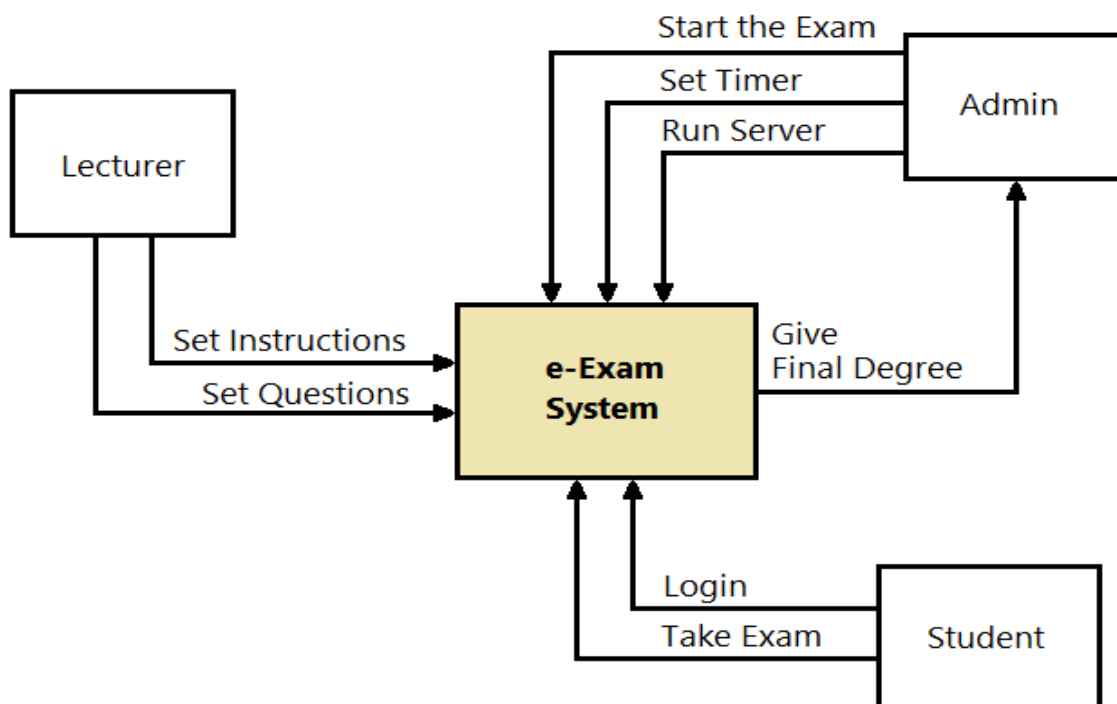
Context Diagram for Online Examination System

Context Diagram (Level-0 DFD)

- The **entire system is shown as a single process**.
- It shows **interaction between the system and external entities** only.
- **No internal details** are shown.

External Entities

- **Student**
- **Admin / Examiner**



3.(a) Describe spiral model in SDLC with diagram.

(b) Discuss the meaning of functionally independent modules. (8+2)?

Ans :- (a) **Spiral Model in SDLC (8 marks)**

Spiral Model – Description

The **Spiral Model** is a **risk-driven, iterative software development model** that combines features of the **Waterfall model, Prototyping model, and Incremental model**.

Development proceeds in a **series of loops (spirals)**, and **each loop represents one phase** of the SDLC.

The **main focus** of the Spiral Model is **risk identification and risk reduction**.

Phases of the Spiral Model

Each loop of the spiral consists of **four main activities**:

I. Planning

- Define objectives, alternatives, and constraints
- Identify requirements and resources

II. Risk Analysis

- Identify potential risks (technical, cost, schedule)
- Reduce risks using prototyping or simulations

III. Engineering (Development & Testing)

- Design, coding, and testing of the selected solution

IV. Evaluation

- Customer evaluates the developed product
- Decision taken to continue with the next spiral

Spiral Model Diagram (Simple Exam Diagram)

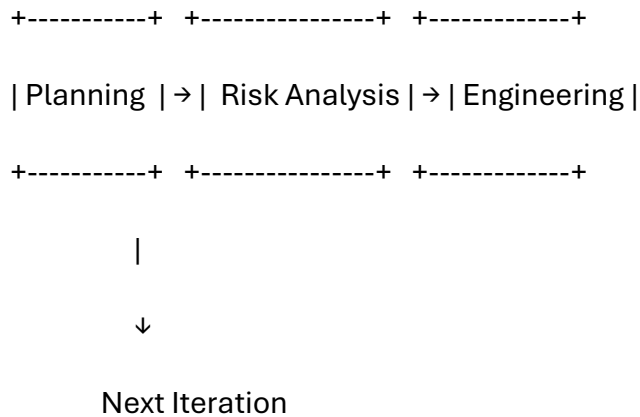
+-----+

| Evaluation |

+-----+

↑

|



(Each loop moves outward, adding more features and reducing risk.)

Advantages

- Best suited for **large and high-risk projects**
- **Early detection of risks**
- Continuous **customer feedback**

Disadvantages

- Complex to manage
 - Not suitable for **small projects**
 - Requires **experienced management**
-

(b) Functionally Independent Modules (2 marks)

A module is said to be **functionally independent** if it has:

- **High cohesion** (performs one specific task)
- **Low coupling** (minimal dependency on other modules)

Meaning

Functionally independent modules are **easy to understand, test, modify, and maintain**, and they improve overall **software quality**.

4. Draw and explain context diagram and 1-level DFD for “Online library Management System”.

Ans :-